



Security Review For Liquid Bitcoin Foundation



Collaborative Audit Prepared For:
Lead Security Expert(s):

Liquid Bitcoin Foundation
defsec
samurai77

Date Audited:
Final Commit:

August 25 - August 27, 2025
025c668

Introduction

This review focuses on the update of the TokenDistributor.sol contract of Liquid Bitcoin's ecosystem.

Scope

Repository: Liquid-Bitcoin/BARD

Audited Commit: 6834fcb0134d15c7e5f9144d0c135b0ae5580b81

Final Commit: 025c6681116ecf4bd7359720670ba568d06a0251

Files:

- contracts/BARD/TokenDistributor.sol

Final Commit Hash

025c6681116ecf4bd7359720670ba568d06a0251

Findings

Each issue has an assigned severity:

- Medium issues are security vulnerabilities that may not be directly exploitable or may require certain conditions in order to be exploited. All major issues should be addressed.
- High issues are directly exploitable security vulnerabilities that need to be fixed.
- Low/Info issues are non-exploitable, informational findings that do not pose a security risk or impact the system's integrity. These issues are typically cosmetic or related to compliance requirements, and are not considered a priority for remediation.

Issues Found

High	Medium	Low/Info
0	0	5

Issues Not Fixed and Not Acknowledged

High	Medium	Low/Info
0	0	0

Issue L-1: VAULT and PAUSER are upper-case despite not actually supposed to

Source: <https://github.com/sherlock-audit/2025-08-lombard-distributor-update-aug-25th/issues/2>

Summary

VAULT and PAUSER are upper-case despite not actually supposed to

Vulnerability Detail

The contract has a few variables like MERKLE_ROOT, VAULT and PAUSER (and more). They are all upper-case, which makes sense for variables like MERKLE_ROOT which are immutable (this pattern is standard for immutable/constant variables):

```
bytes32 public immutable MERKLE_ROOT;
```

However, VAULT and PAUSER are not immutable and they also have setter functions.

Impact

No impact besides potential confusion.

Code Snippet

<https://github.com/sherlock-audit/2025-08-lombard-distributor-update-aug-25th/blob/527cff3113f22868ae8eaaf141b937da00d0819f/BARD/contracts/BARD/TokenDistributor.sol#L61-L78>

Tool Used

Manual Review

Recommendation

Consider not using upper cases for VAULT and PAUSER.

Discussion

555-andrew

Fixed here: <https://github.com/Liquid-Bitcoin/BARD/pull/8/commits/94873031db0c87e24015876f317e7859f8e4dd0c>

samurair77

@555-andrew hey, can I get access to the repo?

defsec

Same as too me If possible, thank you!

samurair77

LGTM

Discussion

555-andrew

Fixed here: <https://github.com/Liquid-Bitcoin/BARD/pull/8/commits/94873031db0c87e24015876f317e7859f8e4dd0c>

samurair77

@555-andrew hey, can I get access to the repo?

defsec

Same as too me If possible, thank you!

samurair77

LGTM

Issue L-2: Malicious users can reroute user funds against their will

Source: <https://github.com/sherlock-audit/2025-08-lombard-distributor-update-aug-25th/issues/3>

Summary

Malicious users can reroute user funds against their will

Vulnerability Detail

The claiming process works in a permissionless way, i.e. Bob can claim for Alice and the outcome will be the same regardless of the caller, thus not problematic. However, when using `claimAndStake()`, the malicious users can influence and change the outcome:

```
function _claimAndStake(
    address _account,
    uint256 _amount,
    bytes32[] calldata _merkleProof,
    uint256 _stakeAmount
) internal {
    if (address(VAULT) == address(0)) revert StakingNotEnabled();
    if (_amount < _stakeAmount) revert WrongStakeAmount();
    _validateClaim(_account, _amount, _merkleProof);

    // Mark as claimed and send the tokens
    hasClaimed[_account] = true;
    if (_amount > _stakeAmount) {
        TOKEN.safeTransfer(_account, _amount - _stakeAmount);
    }
    if (_stakeAmount > 0) {
        TOKEN.safeIncreaseAllowance(address(VAULT), _stakeAmount);
        VAULT.deposit(_stakeAmount, _account);
    }

    emit Claimed(_account, _amount);
}
```

For example, the following can happen (this is just one example scenarios, for example a vice versa case of the below is also possible):

1. Alice simply wants to claim her tokens and does not want to stake them.
2. She calls `claim()` with her merkle proof.

3. Bob sees the transaction in the mempool and copies it, but calls `claimAndStake()` instead, providing `_stakeAmount` to be equal to `_amount`.
4. Outcome is different and now Alice's funds are in the vault. This can be very problematic in some cases such as:
 - there are exit fees in the vault, thus Alice loses funds if she wants her funds back.
 - Alice is not an EOA, but a contract which does not have functionality to withdraw funds from the vault.

Impact

Malicious users can change the desired outcome of the legitimate user, which can in turn cause loss of funds in extreme edge cases.

Code Snippet

<https://github.com/sherlock-audit/2025-08-lombard-distributor-update-aug-25th/blob/527cff3113f22868ae8eaaf141b937da00d0819f/BARD/contracts/BARD/TokenDistributor.sol#L179-L185>

Tool Used

Manual Review

Recommendation

Consider adding access control or if you wish to keep the same design, include `_stakeAmount` in the merkle leaf, however that will also require the regular claim flow to also be changed. An option is to have access control just on `claimAndStake()` as otherwise just claiming for another user is never problematic.

Discussion

555-andrew

Fixed here: <https://github.com/Liquid-Bitcoin/BARD/pull/8/commits/d40fc845cdeda5d5748f65b41926ac0519c8e74a>

samuraii77

Hey, about this one, my recommendation was not perfect and there is still a small grieving vector. Currently, the vice versa scenario is possible:

1. Bob calls `claimAndStake()` in order to stake all of her tokens after claiming.

2. Alice frontruns and calls `claim()` again rerouting the funds into Bob's wallet instead of the intended vault destination.

It is less impactful as Bob could just stake afterwards, so technically can be left as is, but still something to consider fixing.

@555-andrew

lpetroulakis

The issue is partially fixed in [PR#13](#). The team acknowledges and accepts the risk for the second flow.

Discussion

555-andrew

Fixed here: <https://github.com/Liquid-Bitcoin/BARD/pull/8/commits/d40fc845cdeda5d5748f65b41926ac0519c8e74a>

samuraii77

Hey, about this one, my recommendation was not perfect and there is still a small griefing vector. Currently, the vice versa scenario is possible:

1. Bob calls `claimAndStake()` in order to stake all of her tokens after claiming.
2. Alice frontruns and calls `claim()` again rerouting the funds into Bob's wallet instead of the intended vault destination.

It is less impactful as Bob could just stake afterwards, so technically can be left as is, but still something to consider fixing.

@555-andrew

lpetroulakis

The issue is partially fixed in [PR#13](#). The team acknowledges and accepts the risk for the second flow.

Issue L-3: The `pause()` and `unpause()` functions are declared with public visibility instead of external

Source: <https://github.com/sherlock-audit/2025-08-lombard-distributor-update-aug-25th/issues/4>

Summary

The `pause()` and `unpause()` functions are declared with public visibility instead of external

Vulnerability Detail

The contract uses public visibility for the `pause()` and `unpause()` functions at lines 125 and 131 respectively. Since these functions are never called internally within the contract.

Impact

Using public visibility forces the EVM to copy function arguments from calldata to memory, which is an unnecessary operation that increases gas costs.

Code Snippet

```
/**
 * Pause deposit reporting and withdrawal validation.
 */
function pause() public onlyPauser {
    _pause();
}

/**
 * Unpause deposit reporting and withdrawal validation.
 */
function unpause() public onlyOwner {
    _unpause();
}
```

Tool Used

Manual Review

Recommendation

Change the visibility of both functions from public to external.

Discussion

555-andrew

Fixed here: <https://github.com/Liquid-Bitcoin/BARD/pull/8/commits/94873031db0c87e24015876f317e7859f8e4dd0c>

defsec

Fixed with <https://github.com/Liquid-Bitcoin/BARD/commit/94873031db0c87e24015876f317e7859f8e4dd0c>.

Discussion

555-andrew

Fixed here: <https://github.com/Liquid-Bitcoin/BARD/pull/8/commits/94873031db0c87e24015876f317e7859f8e4dd0c>

defsec

Fixed with <https://github.com/Liquid-Bitcoin/BARD/commit/94873031db0c87e24015876f317e7859f8e4dd0c>.

Issue L-4: Inconsistent variable naming convention

Source: <https://github.com/sherlock-audit/2025-08-lombard-distributor-update-aug-25th/issues/7>

Summary

The contract inconsistently changes some variables from uppercase to lowercase while keeping others uppercase.

Vulnerability Detail

The update changes VAULT → vault and PAUSER → pauser (making them appear as regular state variables) while keeping APPROVER, TOKEN, MERKLE_ROOT, and CLAIM_END in uppercase (immutable/constant convention).

Impact

Confusion about variable mutability.

Code Snippet

```
// Inconsistent naming
IERC4626 public vault;           // lowercase (mutable)
address public pauser;           // lowercase (mutable)
address public APPROVER;         // UPPERCASE (mutable??)
// Immutables still uppercase
IERC20 public immutable TOKEN;
bytes32 public immutable MERKLE_ROOT;
```

Tool Used

Manual Review

Recommendation

Follow Solidity naming conventions consistently:

- UPPERCASE for constants and immutables
- camelCase for mutable state variables

Discussion

555-andrew

Fixed here: <https://github.com/Liquid-Bitcoin/BARD/pull/12>

defsec

Fix is confirmed.

Discussion

555-andrew

Fixed here: <https://github.com/Liquid-Bitcoin/BARD/pull/12>

defsec

Fix is confirmed.

Issue L-5: Gas inefficiency from double Keccak256 hashing

Source: <https://github.com/sherlock-audit/2025-08-lombard-distributor-update-aug-25th/issues/9>

Summary

The merkle leaf generation uses unnecessary double keccak256 hashing, wasting gas on every claim verification.

Vulnerability Detail

The code performs `keccak256(bytes.concat(keccak256(...)))` which hashes the data twice. This double hashing provides no additional security benefit while increasing gas costs for every claim.

Impact

Increased gas costs for users.

Code Snippet

```
// Unnecessary double hashing
bytes32 leaf = keccak256(
    bytes.concat(
        keccak256(abi.encode(_account, _amount, ACCOUNT_TYPE_ADDRESS))
    )
);

// Same issue in proof validation
bytes32 leaf = keccak256(
    bytes.concat(
        keccak256(abi.encode(_account, _amount, ACCOUNT_TYPE_BYTES32))
    )
);
```

Tool Used

Manual Review

Recommendation

Remove the outer bytes.concat and one layer of hashing.

Discussion

555-andrew

Originally we were building leaf only from account and amount so the double-hashing was used to prevent "second preimage attacks" (rather "node as leaf attack"). But now the leaf value before hashing is no longer 64 bytes, so, yes, the second hashing is not required. Fixed here: <https://github.com/Liquid-Bitcoin/BARD/pull/12/files>

defsec

Fix is confirmed.

Discussion

555-andrew

Originally we were building leaf only from account and amount so the double-hashing was used to prevent "second preimage attacks" (rather "node as leaf attack"). But now the leaf value before hashing is no longer 64 bytes, so, yes, the second hashing is not required. Fixed here: <https://github.com/Liquid-Bitcoin/BARD/pull/12/files>

defsec

Fix is confirmed.

Disclaimers

Sherlock does not provide guarantees nor warranties relating to the security of the project.

Usage of all smart contract software is at the respective users' sole risk and is the users' responsibility.